

Progressive Streaming Integration Guide - CORRECT Architecture

Date: 2025-10-02 **Status:** ✓ **IMPLEMENTATION COMPLETE** **Correct Architecture:**

MGFDKernel → mgfd_routes.py

⚠ Important Correction

The previous integration guide incorrectly referenced `libs/services/sales_assistant/service.py` and `api/sales_routes.py`, which are **deprecated** and will be removed in the future.

The **actual correct architecture** is: - **Main Service:** `libs/MGFDKernel.py` (MGFDKernel class) - **API Routes:** `api/mgfd_routes.py` (MGFD routes) - **Entry Point:** `main.py` (imports `mgfd_routes`)

Current System Architecture

```
main.py (FastAPI app)
├── api/mgfd_routes.py (MGFD API routes)
│   └── libs/MGFDKernel.py (Main service kernel)
│       ├── UserInputHandler
│       ├── StateManagementHandler
│       ├── PromptManagementHandler
│       ├── KnowledgeManager
│       └── ResponseGenHandler
```

Key Entry Point: `MGFDKernel.process_message()` at [MGFDKernel.py:734-770](#)

Integration Steps (Corrected)

Step 1: Add Progressive Streaming to MGFDKernel

Edit `libs/MGFDKernel.py` and add progressive streaming support:

```
# At the top of MGFDKernel.py, add import
from .services.sales_assistant.progressive_streaming import (
    create_progressive_streaming_service
)

class MGFDKernel:
    def __init__(self, redis_client: Optional[redis.Redis] = None) -> None:
        # ... existing initialization ...

        # Add progressive streaming service initialization
        self.progressive_service = None
```

```

logger.info("MGFDKernel 初始化完成")

def get_progressive_service(self):
    """Get or create progressive streaming service (lazy initialization)"""
    if not self.progressive_service:
        try:
            # Create a minimal service wrapper for compatibility
            from dataclasses import dataclass

            @dataclass
            class ServiceWrapper:
                llm: Any
                milvus_query: Any
                duckdb_query: Any

            # Wrap current components
            wrapper = ServiceWrapper(
                llm=self.llm,
                milvus_query=self.knowledge_manager.milvus_query if hasattr(self.knowledge_manager, 'milvus_query') else None,
                duckdb_query=self.knowledge_manager.duckdb_query if hasattr(self.knowledge_manager, 'duckdb_query') else None
            )

            self.progressive_service = create_progressive_streaming_service(wrapper)
            logger.info("Progressive streaming service initialized")
        except Exception as e:
            logger.error(f"Failed to initialize progressive streaming: {e}")
            self.progressive_service = None

    return self.progressive_service

async def process_message_progressive(
    self,
    session_id: str,
    message: str
) -> AsyncGenerator[str, None]:
    """
    Process message with progressive streaming (5-phase system)

    This is an alternative to process_message() that uses progressive streaming

    Args:
        session_id: Session ID
        message: User message

    Yields:
        SSE-formatted strings with progressive updates
    """
    try:
        # Get or create progressive service
        service = self.get_progressive_service()

        if not service:
            # Fallback to regular processing
            logger.warning("Progressive service not available, using fallback")
            result = await self.process_message(session_id, message, stream=True)

```

```

        yield f"data: {json.dumps(result, ensure_ascii=False)}\n\n"
        return

    # Use progressive streaming
    async for update in service.chat_stream_progressive(message):
        yield update

except Exception as e:
    logger.error(f"Error in progressive streaming: {e}")
    error_response = {
        "type": "error",
        "message": f"處理失敗: {str(e)}",
        "success": False
    }
    yield f"data: {json.dumps(error_response, ensure_ascii=False)}\n\n"

```

Step 2: Add Progressive Endpoint to mgfd_routes.py

Edit `api/mgfd_routes.py` and add the new progressive endpoint:

```

from fastapi import APIRouter, Request
from fastapi.responses import StreamingResponse, JSONResponse
from starlette.responses import StreamingResponse as StarletteStreamingResponse
import logging

router = APIRouter()
logger = logging.getLogger(__name__)

# ... existing imports and mgfd_system initialization ...

@router.post("/chat-progressive")
async def chat_progressive(request: Request):
    """
    Progressive streaming chat endpoint with 5-phase system

    Request body:
    {
        "session_id": "optional_session_id",
        "message": "user message"
    }

    Returns:
    SSE stream with progressive updates
    """
    try:
        data = await request.json()
        message = data.get("message")
        session_id = data.get("session_id")

        if not message:
            return JSONResponse(
                status_code=400,

```

```

        content={"error": "Message is required"}
    )

    # Generate session_id if not provided
    if not session_id:
        import uuid
        session_id = str(uuid.uuid4())

    logger.info(f"Progressive chat request - Session: {session_id}, Message: {message}")

    # Use progressive streaming
    if hasattr(mgfd_system, 'process_message_progressive'):
        return StreamingResponse(
            mgfd_system.process_message_progressive(session_id, message),
            media_type="text/event-stream",
            headers={
                "Cache-Control": "no-cache",
                "Connection": "keep-alive",
                "X-Accel-Buffering": "no"
            }
        )
    else:
        # Fallback to regular processing
        logger.warning("Progressive streaming not available, using fallback")
        result = await mgfd_system.process_message(session_id, message, stream=True)
        return JSONResponse(content=result)

except Exception as e:
    logger.error(f"Error in progressive chat: {e}")
    return JSONResponse(
        status_code=500,
        content={
            "error": "Internal server error",
            "details": str(e)
        }
    )
)

```

Step 3: Update Frontend HTML

Edit `templates/index.html` (or `templates/mgfd_interface.html`) to include progressive streaming resources:

```

<!DOCTYPE html>
<html>
<head>
    <title>MGFD AI Assistant</title>

    <!-- Add progressive streaming CSS -->
    <link rel="stylesheet" href="/static/css/progressive_streaming.css">

    <!-- Existing CSS -->
    <link rel="stylesheet" href="/static/css/styles.css">
</head>

```

```

<body>
  <!-- Progress bar container -->
  <div class="progress-container">
    <div id="progress-bar" class="progress-bar"></div>
  </div>

  <!-- Chat messages container -->
  <div id="chat-messages"></div>

  <!-- User input -->
  <div class="input-container">
    <input type="text" id="user-input" placeholder="輸入訊息...">
    <button id="send-btn">發送</button>
  </div>

  <!-- Scripts -->
  <script src="/static/js/marked.min.js"></script>
  <script src="/static/js/progressive_markdown_renderer.js"></script>
  <script src="/static/js/mgfd_ai.js"></script>
</body>
</html>

```

Step 4: Update Frontend JavaScript

Edit `static/js/mgfd_ai.js` (or create new integration):

```

// Enable progressive streaming mode
let USE_PROGRESSIVE_STREAMING = true; // Feature flag

async function sendMessage() {
  const userInput = document.getElementById('user-input');
  const message = userInput.value.trim();

  if (!message) return;

  // Display user message
  appendUserMessage(message);
  userInput.value = '';

  // Get or generate session ID
  let sessionId = getSessionId();

  if (USE_PROGRESSIVE_STREAMING) {
    // Use progressive streaming
    startProgressiveChat(
      message,
      '/api/mgfd/chat-progressive',
      '#chat-messages',
      '#progress-bar',
      sessionId
    );
  } else {

```

```

        // Use traditional chat
        await sendTraditionalChat(message, sessionId);
    }
}

// Enhanced progressive chat with session ID
function startProgressiveChat(query, endpoint, containerSelector, progressSelector) {
    const renderer = new ProgressiveMarkdownRenderer(
        containerSelector,
        progressSelector
    );

    // Reset renderer
    renderer.reset();

    // Make POST request with session_id
    fetch(endpoint, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({
            message: query,
            session_id: sessionId
        })
    })
    .then(response => {
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        // Read SSE stream
        const reader = response.body.getReader();
        const decoder = new TextDecoder();

        return readStream(reader, decoder, renderer);
    })
    .catch(error => {
        console.error('Progressive chat error:', error);
        renderer.handleError(`請求失敗: ${error.message}`);
    });
}

async function readStream(reader, decoder, renderer) {
    let buffer = '';

    while (true) {
        const { value, done } = await reader.read();

        if (done) break;

        buffer += decoder.decode(value, { stream: true });

        // Process complete SSE messages
        const lines = buffer.split('\n\n');
    }
}

```

```

    buffer = lines.pop(); // Keep incomplete message

    for (const line of lines) {
        if (line.startsWith('data: ')) {
            const jsonString = line.substring(6);

            if (jsonString) {
                try {
                    const data = JSON.parse(jsonString);
                    handleProgressiveUpdate(data, renderer);
                } catch (e) {
                    console.error('JSON parse error:', e, jsonString);
                }
            }
        }
    }
}

function handleProgressiveUpdate(data, renderer) {
    console.log('Progressive update:', data.type, data);

    switch (data.type) {
        case 'progress':
            renderer.updateProgress(
                data.phase,
                data.message,
                data.progress
            );
            break;

        case 'phase_result':
            console.log(`Phase ${data.phase} complete:`, data.data);
            break;

        case 'markdown_token':
            renderer.addToken(data.token);
            break;

        case 'complete':
            renderer.complete();
            break;

        case 'error':
            renderer.handleError(data.message);
            break;

        default:
            console.warn('Unknown message type:', data.type);
    }
}

// Session ID management
function getSessionId() {
    let sessionId = sessionStorage.getItem('mgfd_session_id');

```

```

    if (!sessionId) {
        sessionId = generateUUID();
        sessionStorage.setItem('mgfd_session_id', sessionId);
    }

    return sessionId;
}

function generateUUID() {
    return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, function(c) {
        const r = Math.random() * 16 | 0;
        const v = c == 'x' ? r : (r & 0x3 | 0x8);
        return v.toString(16);
    });
}

// Initialize
document.addEventListener('DOMContentLoaded', function() {
    const sendBtn = document.getElementById('send-btn');
    const userInput = document.getElementById('user-input');

    sendBtn.addEventListener('click', sendMessage);
    userInput.addEventListener('keypress', function(e) {
        if (e.key === 'Enter') {
            sendMessage();
        }
    });
});
});

```

Configuration

No changes needed to existing configuration files. The progressive streaming system uses the same configuration as the existing MGFDKernel.

Optional: Add feature flag to `config.py`:

```

# Progressive Streaming Configuration
PROGRESSIVE_STREAMING_ENABLED = True
PROGRESSIVE_MAX_CONTEXT_TOKENS = 100000

```

Testing

Manual Testing

1. **Start the server:** `bash python main.py`
2. **Open browser:** `http://localhost:8001`
3. **Test progressive streaming:**

4. Enter a query in the chat interface
5. Watch for phase-by-phase progress updates
6. Observe token-by-token markdown rendering

7. Test queries:

8. Simple: "APX819 規格"
9. Comparison: "比較 APX819 和 APX839"
10. Recommendation: "推薦適合遊戲的筆電"

API Testing with cURL

```
# Test progressive endpoint
curl -X POST http://localhost:8001/api/mgfd/chat-progressive \
-H "Content-Type: application/json" \
-d '{"message": "APX819 規格", "session_id": "test-123"}' \
--no-buffer
```

Performance Benchmarks

Same as original implementation:

Metric	Target	Expected
First update	< 50ms	~20ms
Phase 1 complete	< 500ms	~300-500ms
First token	< 2s	~1.5s
Full response	5-8s	5-7s

Troubleshooting

Issue: "Progressive service not available"

Cause: Progressive streaming components not properly initialized

Solution: Check that all phase modules are in the correct location:

```
ls -la libs/services/sales_assistant/phase*.py
ls -la libs/services/sales_assistant/progressive_streaming.py
```

Issue: "Module import error"

Cause: Import path mismatch due to different directory structure

Solution: Adjust import paths in MGFDKernel.py:

```
# Use absolute imports from project root
from libs.services.sales_assistant.progressive_streaming import ...
```

Issue: "LLM not initialized"

Cause: MGFDKernel's LLM initialization failed

Solution: Check LLM initialization in [MGFDKernel.py:75-103](#)

Migration Notes

Differences from sales_assistant/service.py

The deprecated sales_assistant/service.py will be removed. Key differences:

Old (Deprecated)	New (Current)
SalesAssistantService	MGFDKernel
api/sales_routes.py	api/mgfd_routes.py
service.chat_stream()	MGFDKernel.process_message()

Why This Architecture?

- **Unified System:** MGFDKernel is the single source of truth
 - **Modular Design:** All handlers (User Input, State, Knowledge, etc.) are properly separated
 - **State Management:** Built-in session and state management
 - **Future-Proof:** Services folder will be deprecated
-

Verification Checklist

- ☐ Progressive streaming modules in libs/services/sales_assistant/
 - ☐ MGFDKernel.process_message_progressive() method added
 - ☐ /api/mgfd/chat-progressive endpoint in mgfd_routes.py
 - ☐ Frontend includes progressive_markdown_renderer.js and CSS
 - ☐ Frontend JavaScript uses correct API endpoint
 - ☐ Manual testing completed
 - ☐ Browser dev console shows no errors
 - ☐ Progressive updates display correctly
-

Summary

The progressive streaming system has been **successfully implemented** and is ready for integration with the **correct architecture** (MGFDKernel + mgfd_routes.py).

Next Action: Follow Steps 1-4 above to integrate into your MGFDKernel-based system.

Document Status: ✓ Complete (Corrected) **Integration Status:** Pending **Tested:** Pending manual testing

Last Updated: 2025-10-02 **Version:** 1.0.1 (Corrected)